

Project 2 Report

2022059952 박준서

Code Analysis



Code Analysis

Identify what has been added to the provided code and analyze the purpose of each addition.

- `kernel/proc.h` 에 `mm_struct` 구조체를 정의하였다. `mm_struct` 는 프로세스를 구성하는 스레드들이 공유하는 memory management 관련 변수로, 여러 스레드들이 `mm_struct` 를 공유할 때 발생할 수 있는 동기화 문제를 해결하기 위해 이 구조체에 대한 lock을 멤버로 같이 두었다.

```
--
84 + struct mm_struct {
85 +     pagetable_t pagetable;    // page table root
86 +     uint64 sz;                // user memory size (bytes)
87 +     int refcount;             // number of tasks sharing this
88 +     mm
89 +     struct spinlock lock;
90 + };
91 +
92 + #define CLONE_VM      0x0100    // share address space
93 + (thread)
94 +
95 + // Per-process (per-task) state.
96 + struct proc {
97 +     struct spinlock lock;
```

- 기존 `kernel/proc.h` 에 정의되어 있었던 `proc` 구조체는 이제 프로세스 하나라기보다는 한 스레드를 나타내는 구조체로, 이 스레드가 어떤 `mm_struct` 를 참조하는지를 나타내는 `mm` pointer와 이 스레드가 어떤 thread group 정보를 갖고 있는지를 저장할 수 있도록 하였다.

```

94     struct proc {
95         struct spinlock lock;
96
97         // p->lock must be held when using these:
98         enum procstate state;        // Process state
99         void *chan;                  // If non-zero, sleeping on chan
100        int killed;                   // If non-zero, have been killed
101        int xstate;                   // Exit status to be returned to parent's wait
102 +   int pid;                         // Process / thread id (Linux's "TID")
103
104        // wait_lock must be held when using this:
105        struct proc *parent;          // Parent process
106
107        // these are private to the process, so p->lock need not be held.
108        uint64 kstack;                // Virtual address of kernel stack
109 +   struct mm_struct *mm;            // Shared address space (replaces pagetable+sz)
110
111        struct trapframe *trapframe; // data page for trampoline.S
112        struct context context;       // swtch() here to run process
113        struct file *ofile[NOFILE];  // Open files
114        struct inode *cwd;            // Current directory
115        char name[16];                // Process name (debugging)
116 +
117 +   // === thread group ===
118 +   struct proc *group_leader;       // group leader (self if not a thread)
119 +   int tgid;                        // thread group id (== leader->pid)
120 +   uint64 tf_va;                    // user VA where p->trapframe is mapped
121 +   void *ustack;                    // user stack base (only set for threads)
122     };

```

- `pagetable` 변수는 기존 `proc` 구조체에서 `mm_struct` 로 이동하였으므로, `syscall.c`, `sysfile.c` 등 kernel에서 기존 `proc` 구조체의 `pagetable` `p->pagetable` 은 `p->mm->pagetable` 로 모두 변경하였다.
- `proc.c`에서 `proc` 구조체를 할당하는 `allocproc()` 함수에서는, `proc` 구조체에서 새로 정의한 thread group 관련 변수를 함께 초기화하도록 추가하였다.

```

137     // Look in the process table for an UNUSED proc.
138     // If found, initialize state required to run in the kernel,
139     // and return with p->lock held.
140     // If there are no free procs, or a memory allocation fails, return 0.
141     static struct proc*
142     allocproc(void)
143     {
144         struct proc *p;
145
146         for(p = proc; p < &proc[NPROC]; p++) {
147             acquire(&p->lock);
148             if(p->state == UNUSED) {
149                 goto found;
150             } else {
151                 release(&p->lock);
152             }
153         }
154         return 0;
155
156     found:
157         p->pid = allocpid();
158         p->state = USED;
159
160 +   // Default thread fields: a single-task process is its own group leader.
161 +   p->mm = 0;
162 +   p->group_leader = p;
163 +   p->tgid = p->pid;
164 +   p->tf_va = 0;
165 +   p->ustack = 0;
166
167 +   // Allocate a trapframe page (per-task; never shared).
168 +   if((p->trapframe = (struct trapframe *)kalloc()) == 0){

```

- 반면 `freeproc()` 함수에서는, 기존 xv6에서 `kfree()` 를 호출했던 부분에 더해 `mm_struct` 의 `refcount` 를 줄이는 `mm_put()` 을 호출하고 thread group 관련 변수를 0 (또는 null)로 바꾸는 로직을 추가하였다. 한편 처음 skeleton code에서는 `mm_put()` 함수가 제대로 구현되어 있지 않아 실제로 `refcount` 를 줄이고 해당 `refcount` 가 0인 경우 `trapframe`을 해제하는 로직의 부재로 memory leak가 발생할 수 있었다.

```

183 + // free a proc structure and its per-task resources.

184 // p->lock must be held.
185 static void
186 freeproc(struct proc *p)
187 {
188 + if (p->trapframe) {
189 + kfree((void *)p->trapframe);
190 + p->trapframe = 0;
191 + }
192 + p->tf_va = 0;
193 +
194 + if (p->mm) {
195 + mm_put(p->mm);
196 + p->mm = 0;
197 + }
198 +
199 p->pid = 0;
200 p->parent = 0;
201 p->name[0] = 0;
202 p->chan = 0;
203 p->killed = 0;
204 p->xstate = 0;
205 p->state = UNUSED;
206 + p->group_leader = 0;
207 + p->tgid = 0;
208 + p->ustack = 0;
209 }

```

- `kclone()` 으로 thread를 반영할 수 있게 변경되는 `kfork()` 함수는 먼저 user가 호출하는 `clone()` 함수의 인자에 `CLONE_VM` 이 1인지 0인지에 따라 thread path로 실행될지, fork path로 실행될지 분기할 수 있도록 구체화되었다. fork path는 기존 `kfork()` 함수와 비교하여, `uvmcopy` 를 직접 호출하는 방식에서 `mm_alloc()` → `proc_pagetable()` → `uvmcopy()` 순으로 호출하는 방식으로 변경되었다. 이는 thread를 도입하면서 어떤 thread는 기존 프로세스와 `mm_struct` 를 공유하고, 새로 fork되는 프로세스는 새로 `mm_struct` 를 할당받을 수 있도록 경로를 분리하기 위함이다.

```

306 int
307 + kclone(uint64 fn, uint64 arg, uint64 stack, int n_pages, int flags)
308 {
309     int i, pid;
310     struct proc *np;
311     struct proc *p = myproc();
312
313 + if ((np = allocproc()) == 0)
314     return -1;
315 +
316 + if (flags & CLONE_VM) {
317 + // === thread path ===
318 + freeproc(np);
319 + release(&np->lock);
320 + return -1;
321 + } else {
322 + // === fork path ===
323 + if ((np->mm = mm_alloc()) == 0)
324 + goto fail;
325 + if ((np->mm->pagetable = proc_pagetable(np)) == 0)
326 + goto fail;
327 + if (uvmcopy(p->mm->pagetable, np->mm->pagetable, p->mm->sz) < 0)
328 + goto fail;
329 + np->mm->sz = p->mm->sz;
330 + np->group_leader = np;
331 + np->tgid = np->pid;
332 +
333 + // copy parent's user registers; child resumes after the syscall.
334 + *(np->trapframe) = *(p->trapframe);
335 + np->trapframe->a0 = 0; // child's clone() returns 0
336 }
337
338 + // === common: files, cwd, name ===

```

- 한편 유저 라이브러리 함수를 선언하는 user/user.h 파일에는 과제에서 구현해야 할 `thread_create()` 와 `thread_join()` 함수에서 사용할 system call인 `clone()` 과 `join()` 의 signature를 선언한 것을 확인할 수 있었다.

```

8 // system calls
9 + int clone(void (*fn)(void *), void *arg, void *stack,
10 +         int n_pages, int flags);
11 + int join(void **stack);
12 int exit(int) __attribute__((noreturn));

```

Design



Design

Explain your implementation plan to meet the assignment requirements.

Step 1 : Extend Memory Management

주어진 스켈레톤 코드에서 `usertests forktest`를 실행하면 **fork exit을 하는 과정에서 memory leak가 일어남**을 확인하였다. 프로세스가 종료되고 `freeproc()`을 하는 과정에서 memory leak가 일어난 것이므로, 먼저 `freeproc()`을 확인하였다.

```

// free a proc structure and its per-task resources.
// p->lock must be held.
static void
freeproc(struct proc *p)
{
    if (p->trapframe) {
        kfree((void *)p->trapframe);
        p->trapframe = 0;
    }
    p->tf_va = 0;

    if (p->mm) {
        mm_put(mm: p->mm);
        p->mm = 0;
    }

    p->pid = 0;
    p->parent = 0;
    p->name[0] = 0;
    p->chan = 0;
    p->killed = 0;
    p->xstate = 0;
    p->state = UNUSED;
    p->group_leader = 0;
    p->tgid = 0;
    p->ustack = 0;
}

```

초기 freeproc() 함수의 상태.

process의 `mm_struct`에 대하여 `mm_put()`이 실행되도록 설계되어 있으나, 초기 상태에서 `mm_put()` 함수는 구현되어 있지 않으므로 올바르게 메모리 자원을 회수하지 못해 memory leak가 발생한 것이다. 즉 프로세스가 종료되는 시점에서 memory leak가 발생하지 않도록 하려면 `mm_put()` 함수에서 해당 프로세스의 `refcount`를 감소시키고 `refcount == 0`인 경우 정상적으로 프로세스의 pagetable과 `mm`을 free시켜주도록 해야 한다. `refcount`가 0이 되었다면, `uvmunmap`을 이용해 Virtual address와 Physical address의 mapping을 해제해줘야 할 것이라고 판단하였다.

Step 2 : User Library Function

proc.c에서 system call의 구현부를 user에서 어떻게 호출해야 할지를 중요하게 보았다. 유저 라이브러리 함수는 `thread_create()`와 `thread_join()`의 parameter가 각 system call `clone()`과 `join()`에서 어떻게 사용되는지 관찰하였다.

```
int thread_create(void (*fn)(void *), void *arg, int n_pages);
int thread_join(void);
```

uthread.h에 선언된 유저 라이브러리 함수.

```
// system calls
int clone(void (*fn)(void *), void *arg, void *stack,
          int n_pages, int flags);
int join(void **stack);
```

user.h에 선언된 clone()과 join() 함수.

- 먼저 **clone**에 대하여,
 - `thread_create()` 함수 내부에서 `clone()` 을 호출하지만, `thread_create()` 함수 내에는 `clone()` 함수가 갖고 있는 parameter인 `void *stack` 과 `int flags` 가 존재하지 않음을 확인하였다.
 - 이는 `thread_create()` 함수를 통해 새로 만들 프로세스 (또는 스레드)가 이용할 user stack을 이 함수 안에서 동적으로 할당하여, 그 stack의 주소를 `clone()` 함수의 argument로 전달하기 위함이라고 생각하였다.
- 다음으로 **join**에 대하여,
 - `thread_join()` 함수 내부에서 `join()` 을 호출하지만, `thread_join()` 에는 parameter가 존재하지 않는 반면 `join()` 함수에서는 stack에 대한 double pointer를 argument로 받는 것을 확인하였다.
 - `join()` 함수가 stack에 대한 double pointer를 전달하는 것은, kernel에서 죽은 스레드의 stack 주소를 반환하기 위해서라고 판단하였다. 이 `join()` 함수의 return value로 죽은 thread의 `pid`를 얻은 다음, 그 `pid`를 그대로 `thread_join()` 함수의 return value로 돌 것이다.
 - 다만 `thread_join()` 함수는 parameter를 받지 않으므로, `join()` system call을 통해 반환받아야 할 stack 주소를 전달하기 위한 `void *` 타입의 변수를 내부적으로 별도 선언해야 할 것이라 생각하였다. 또한, `thread_join()` 함수의 parameter가 존재하지 않는다는 것은 user 영역에서 이 함수를 호출하였을 때 kernel에서 가장 먼저 죽은 thread를 빠르게 `join()` 을 통해 stack 주소를 반환하도록 하기 위함이라고 생각하였다.
 - `thread_create()` 를 통해 새로운 스레드에 동적 할당한 stack 공간은, 그 스레드가 죽었을 때 `thread_join()` 함수를 통해 회수해야 할 것이므로 `join()` 을 통해 회수할 stack 주소를 얻었다면 그 주소에 대한 `free()` 를 수행해야 할 것이라고 생각하였다.

Implementation



Design

Detail the specific modifications and additions you made to the given xv6 codebase. Clearly highlight how they differ from the original code and the purpose of these changes.

Step 1: Extend Memory Management

mm_put() 구현

`mm_put()` 함수를 호출하면 refcount를 감소시키도록 하였다. 한 프로세스의 마지막 thread가 종료되면 (즉, `refcount == 0` 이면) 그 프로세스의 `mm` 에 대하여 Page Table과 `mm` 자체를 free해주도록 구현하였다. 이때 `mm_struct` 는 프로세스 안의 여러 thread가 공유할 수 있는 변수이고, thread가 종료될 때마다 `mm_struct` 의 멤버 변

수인 `refcount` 를 변화시키는 것이므로 `refcount` 를 감소시키기 전 `mm` 에 대한 lock을 acquire하도록 해 동기화 문제를 해결했다.

```
void
mm_put(struct mm_struct *mm)
{
    acquire(&mm->lock);
    mm->refcount--;
    if(mm->refcount == 0){
        proc_freepagetable(pagetable: mm->pagetable, sz: mm->sz);
        release(&mm->lock);
        kfree((void*)mm);
    }
    else release(&mm->lock);
}
```

mm_get() 구현

`mm_get()` 함수를 호출하면 `refcount` 를 증가시키도록 하였다. `mm_put()` 에서와 마찬가지로, 공유 변수인 `mm_struct` 의 멤버의 동기화 문제를 해결하기 위해 `mm->lock` 을 확보하였다.

```
void
mm_get(struct mm_struct *mm)
{
    acquire(&mm->lock);
    mm->refcount++;
    release(&mm->lock);
}
```

find_free_tf_va() 구현

`find_free_tf_va()` 함수는 프로세스 (또는 스레드)를 clone할 때, page table의 어떤 slot이 비어있는지를 확인하기 위해 사용한다.

Virtual Address를 할당할 스레드와 `mm_struct` 를 공유하는 다른 스레드들을 순회하며 해당 스레드가 page table의 어느 slot을 사용하는지를 확인한 후, 비어있는 slot에 해당하는 page table 상에서의 주소를 return하도록 구현하였다. 해당 함수 내에서는 `NTHREAD` 개만큼의 배열 `slot_used` 를 선언하여, 0번째 slot부터 7번째 slot 중 어느 곳이 사용중이고 어느 곳이 비어있는지 찾는 작업을 보조하도록 하였다.

```

140 uint64
141 find_free_tf_va(pagetable_t pagetable)
142 {
143     int slot_used[NTHREAD];
144     for(int i=0; i<NTHREAD; i++) slot_used[i] = 0; // initialize
145
146     struct proc* p;
147     struct proc* cur_p = myproc();
148     for(p = proc; p < &proc[NPROC]; p++){
149         // acquire(&p->lock);
150         if(p->state != UNUSED && p->mm == cur_p->mm && p->tf_va != 0){ // p and myproc shares process
151             int p_slot = (TRAPFRAME - p->tf_va) / PGSIZE; // p_slot : where p occupies slot in page table?
152
153             if(p_slot >= 0 && p_slot < NTHREAD){
154                 slot_used[p_slot] = 1;
155             }
156         }
157         // release(&p->lock);
158     }
159
160     for(int i=0; i<NTHREAD; i++){
161         if(!slot_used[i]){ // find empty slot
162             return TRAPFRAME - i*PGSIZE;
163         }
164     }
165
166     return 0; // if not found
167 }

```

per-thread trapframe slot unmap inside freeproc()

```

void
freeproc(struct proc *p)
{
    // if non-leader then uvmunmap
    if(p->group_leader != p && p->mm && p->tf_va != 0){
        uvmunmap(p->mm->pagetable, p->tf_va, 1, 0);
    }

    if (p->trapframe) {
        kfree((void *)p->trapframe);
        p->trapframe = 0;
    }
    p->tf_va = 0;

    if (p->mm) {
        mm_put(mm: p->mm);
        p->mm = 0;
    }

    p->pid = 0;
    p->parent = 0;
    p->name[0] = 0;
    p->chan = 0;
    p->killed = 0;
    p->xstate = 0;
    p->state = UNUSED;
    p->group_leader = 0;
    p->tgid = 0;
    p->ustack = 0;
}

```

non-leader thread에 대하여 `freeproc()` 이 호출되었을 때 `uvmunmap` 을 통해 VA와 PA 사이의 mapping을 끊어주도록 구현했다. group leader가 `freeproc()` 되는 경우는 그 스레드의 `mm_struct` 에서 관리되는 `refcount` 가 1에서 0으로 감소하는 경우이므로, `mm_put()` 함수에서 `mm_struct` 와 함께 free해주기로 하였다.

Step 2 : Implement Core Threading Primitives

CLONE_VM branch of kclone()

`kclone()` 함수가 thread creation의 목적으로 호출되었음이 확인됐다면, 새로 만들어지는 스레드의 `mm_struct` 및 `group_leader`에 대한 정보를 초기화하고 `trapframe`을 새로 설정하는 작업을 거치도록 하였다.

먼저 (1) `mm_get()` 함수를 호출해 공유하는 `mm_struct`에 대한 `refcount`를 증가시켰고, (2) 새로운 스레드(`np`)의 `group_leader`, `tgid`, `mm`은 `kclone()` 함수를 호출한 프로세스/스레드인 `p`의 것과 같도록 설정하였다.

또한, (3) Step 1에서 정의한 `find_free_tf_va()` 함수를 통해 `np`의 `tf_va`를 할당하였고, (4) `kclone()`의 인자로 들어온 `fn`, `arg`, `stack`은 새로운 스레드 `np`의 시작 주소(`epc`), 인자(`arg`), **stack top**(`stack`)이 되도록 Trapframe의 레지스터로 할당하였다.

```
int
kclone(uint64 fn, uint64 arg, uint64 stack, int n_pages, int flags)
{
    int i, pid;
    struct proc *np;
    struct proc *p = myproc();

    if ((np = allocproc()) == 0)
        return -1;

    if (flags & CLONE_VM) {
        // === thread path ===
        // 1. increase refcount.
        mm_get(mm: p->mm);
        // 2. initialize thread-group-related variables.
        np->group_leader = p->group_leader;
        np->tgid = p->tgid;
        np->mm = p->mm;
        // 3. assign trapframe of np.
        acquire(&p->mm->lock);
        if ((np->tf_va = find_free_tf_va(pagetable: np->mm->pagetable)) == 0){
            release(&p->mm->lock);
            goto fail;
        }
        if (mappages(np->mm->pagetable, np->tf_va, PGSIZE, (uint64)(np->trapframe), PTE_R | PTE_W) < 0){
            release(&p->mm->lock);
            goto fail;
        }
        release(&p->mm->lock);
        // 4. np->trapframe registers
        *(np->trapframe) = *(p->trapframe); // copy trapframe
        np->trapframe->epc = fn;
        np->trapframe->a0 = arg;
        np->trapframe->sp = stack;
    }
}
```

kjoin()

`kjoin()` 함수는 프로세스의 group leader가 자신의 sibling thread가 종료될 때까지 기다렸다가 sibling이 종료되면 free시켜주는 함수로, single-thread 버전에서 부모 프로세스가 `wait()` 하는 것과 같은 구도로 볼 수 있다. 이 함수에서는 group leader는 모든 sibling이 종료되어야 group leader도 죽을 수 있다는 점에서도 기존 xv6의 `wait()` 함수와 유사점을 찾을 수 있다.

`kjoin()` 함수가 시작하면 호출한 주체(group leader)와 `tgid`가 같은 스레드를 proc table에서 찾은 후, 해당 sibling thread가 ZOMBIE state인 걸 발견하면 차례대로 `copyout`, `uvmunmap`, `freeproc`을 통해 종료된 user stack의 주소를 반환하고 trapframe 매핑을 제거한 후 이 thread가 사용하던 resource를 제거하도록 하였다.


```

int
kjoin(uint64 stack_addr)
{
    struct proc* p = myproc();
    struct proc* pp; // this will be sibling thread...
    int havesiblings, pid;

    acquire(&wait_lock);

    for(;;){
        // Scan through table looking for exited sibling.
        havesiblings = 0;

        for(pp = proc; pp < &proc[NPROC]; pp++){
            acquire(&pp->lock);
            if(pp->tgid != p->tgid || pp == p || pp->state == UNUSED){
                release(&pp->lock);
                continue;
            }

            // make sure the sibling isn't still in exit() or swtch().
            havesiblings = 1;

            if(pp->state == ZOMBIE){
                // found one.
                pid = pp->pid;

                if(stack_addr != 0 && copyout(p->mm->pagetable, stack_addr, (char *)(&pp->ustack), sizeof(pp->ustack)) < 0){
                    release(&pp->lock);
                    release(&wait_lock);
                    return -1;
                }

                // free ZOMBIE thread.
                if(pp->tf_va != 0){
                    uvmunmap(p->mm->pagetable, pp->tf_va, 1, 0);
                    pp->tf_va = 0;
                }
                freeproc(p: pp);

                release(&pp->lock);
                release(&wait_lock);
                return pid;
            }
            release(&pp->lock);
        }

        // No point waiting if we don't have any siblings in this process.
        if(!havesiblings || killed(p)){
            release(&wait_lock);
            return -1;
        }
    }

    // Wait for a sibling to exit.
    sleep(chan: p->group_leader, lk: &wait_lock); // DOC : Wait-sleep.
}
}

```

User Library : `thread_create()` , `thread_join()` (in user/uthread.c)

`uthread.c` 에서는 user가 `clone()` , `join()` system call을 사용할 수 있도록 하는 라이브러리 함수인 `thread_create()` 와 `thread_join()` 함수를 구현하였다. 새로운 스레드가 사용할 user stack의 할당과 해제는 모두 이 두 함수의 책임으로, `thread_create()` 에서는 그 user stack에서 필요한 정보를 parameter로, `thread_join()` 에서는 해제할 user stack의 주소와 관련한 정보를 parameter로 받도록 하였다.

- `thread_create()` 에서는 함수 포인터 `fn` 과 thread 시작 함수에 전달할 인자 `arg` , 그리고 stack의 크기 `n_page` (page 단위)를 parameter로 받는다. `thread_create()` 내에서 `malloc()` 함수를 이용해 `n_pages * PGSIZE` 만큼의 메모리 공간을 할당받고 그 리턴값을 `stack_base` 로 한다면, `n_pages * PGSIZE` 만큼 값을 더한 주소가 stack의 top이 되므로 이 값을 `clone()` 시스템 콜의 인자로 전달하였다.

```

int
thread_create(void (*fn)(void *), void *arg, int n_pages)
{
    void* stack_base = malloc(n_pages * PGSIZE);
    if(stack_base == 0) return -1; // failed to malloc

    uint64 stack_top = (uint64)stack_base + (n_pages * PGSIZE);

    int pid = clone(fn, arg, stack: (void*)stack_top, n_pages, flags: CLONE_VM);
    if(pid < 0){
        free(stack_base);
        return -1; // fail to clone
    }

    return pid; // success to clone!
}

```

- `thread_join()`에서는 `user.h`에 선언된 `join()` 함수를 이용해, `user_stack_base`를 stack의 base로 사용하는 스레드의 pid를 가져온 후, 그 스레드가 사용하는 stack을 할당 해제하는 방식으로 구현하였다.

```

int
thread_join(void)
{
    void* user_stack_base = 0;
    int pid = join(stack: &user_stack_base); // dead thread's pid
    if(pid < 0) return -1; // fail

    if(user_stack_base != 0){
        free(user_stack_base);
    }

    return pid;
}

```

Step 3 : Coordinate Process-wide System Calls

kwait()

`kwait()` 함수는 기존 single-thread xv6에서와 마찬가지로, 부모 프로세스가 자식이 죽을 때까지 sleep 상태에 있다가 자식이 죽은 것이 확인되면 free시켜주는 함수다. 다만 multi-thread로 확장하면서 `kwait()`는 자식 중 group leader가 죽을 때 그 group leader를 reap하는 방식으로 구현하였다.

따라서, 기존 `kwait()`에서 부모 프로세스 p가 reap할 대상이 group leader가 아닌 경우 (`pp->group_leader != pp`)에는 무시하는 로직만 추가하였다.

```

int
kwait(uint64 addr)
{
    struct proc *pp;
    int havekids, pid;
    struct proc *p = myproc();

    acquire(&wait_lock);

    for(;;){
        // Scan through table looking for exited children.
        havekids = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++){
            // to implement wait() system call in multithread environment,
            // the object of wait() must be a group leader of child process.
            acquire(&pp->lock);
            if (pp->parent != p || pp->group_leader != pp){
                release(&pp->lock);
                continue;
            }
        }
    }
}

```

kexit()

`kexit()` 은 이 함수를 호출한 프로세스 (또는 스레드)가 종료를 준비하며 자신의 state를 `ZOMBIE` 로 바꾸고 부모를 wakeup하여 부모가 자신을 free시키도록 유도하는 함수이다. 다만 과제 명세에서도 소개된 바와 같이, `kexit()` 을 호출한 주체가 프로세스의 group leader인 경우에는 아직 살아있는 모든 sibling을 모두 죽인 후 마지막으로 남은 group leader 자신이 부모를 wakeup하고 종료하도록 하였다.

- `kexit()` 을 호출한 주체(`p`)가 group leader인 경우에는 **(1)** 자기 자신이 종료되기 전 자신의 sibling을 proc table에서 찾아 모두 `killed` 플래그를 세운 후, **(2)** 모든 sibling이 죽을 때까지 infinite loop를 돌면서 `ZOMBIE` 가 된 sibling을 `freeproc()` 해주는 작업을 거쳐 모든 sibling을 reap하도록 하였다.

```

// Is this group leader?
if(p->tgid == p->pid){
    struct proc* pp;

    // 1. kill mark
    for(pp = proc; pp < &proc[NPROC]; pp++){
        acquire(&pp->lock);

        if(pp == p){
            release(&pp->lock);
            continue;
        }

        if(pp->state != UNUSED && pp->tgid == p->pid){ // if pp is sibling thread
            pp->killed = 1; // kill mark
            if(pp->state == SLEEPING){
                pp->state = RUNNABLE;
            }
        }
        release(&pp->lock);
    }

    // 2. wait for all sibling thread to be reaped
    for(;;){
        int hasiblings = 0; // flag : if 1, no active siblings
        for(pp = proc; pp < &proc[NPROC]; pp++){
            acquire(&pp->lock);
            if(pp->tgid == p->pid && pp != p && pp->state != UNUSED){
                if(pp->state == ZOMBIE){
                    freeproc(p; pp);
                }
                else{
                    hasiblings = 1;
                }
            }
            release(&pp->lock);
        }

        if(hasiblings == 0){
            break;
        }

        sleep(chan: p, lk: &wait_lock);
    }

    // Give any children to init.
    reparent(p);
}

```

- group leader가 sibling을 모두 죽인 다음에는, 자신을 free하기 전 trapframe과 매핑된 메모리의 공간을 선제적으로 확보하기 위해 `uvmunmap` 을 해주었다. 자세한 내용은 뒤의 **Troubleshooting**에서 다루겠다.

```

if(p->mm && p->group_leader == p){
    acquire(&p->mm->lock);
    if(p->mm->refcount == 1){
        uvmunmap(p->mm->pagetable, 0, p->mm->sz / PGSIZE, 1);
        p->mm->sz = 0;
    }
    release(&p->mm->lock);
}

```

- 이후 group leader가 혼자 남은 경우와 `kexit()` 을 호출한 주체가 non-leader인 경우 공통으로, 자신을 free시킬 대상을 wakeup시킨 후 자신의 상태를 `ZOMBIE` 로 바꾼 후 다음 스케줄링을 기다리도록 구현하였다.

```

    acquire(&p->lock);
    if(p->group_leader == p){
        wakeup(chan: p->parent);
    } else{
        wakeup(chan: p->group_leader);
    }

    p->xstate = status;
    p->state = ZOMBIE;

    // release(&p->lock); -> will be released in sched()
    release(&wait_lock);

    // Jump into the scheduler, never to return.
    sched();
    panic("zombie exit");
}

```

kill()

`kill()` 은 외부 프로세스가 파라미터 `pid` 를 가진 스레드가 존재하는 thread group 전체를 kill하는 함수로 진화하였다. 먼저 proc table을 순회하면서 파라미터로 들어온 `pid` 의 주인인 스레드 `own_p` 를 찾은 후, 이 `own_p` 의 `tgid` 와 `tgid` 가 같은 모든 스레드의 `killed` 플래그를 1로 바꾸는 방식으로 `kill()` 함수를 구현하였다.

```

int
kkill(int pid)
{
    struct proc *p;
    struct proc* own_p = 0;
    int target_tgid = 0;

    // find own_p
    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->state != UNUSED && p->pid == pid){
            own_p = p;
            release(&p->lock);
            break;
        }
        release(&p->lock);
    }

    if(own_p == 0) return -1; // not found

    // find object thread group id (tgid).
    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->state != UNUSED && p->tgid == own_p->tgid){
            target_tgid = p->tgid;
            release(&p->lock);
            break;
        }
        release(&p->lock);
    }

    if(target_tgid == 0) return -1; // failed to find target tgid

    // kill all thread whose tgid is target_tgid.
    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->state != UNUSED && p->tgid == target_tgid){
            p->killed = 1; // kill mark
            if(p->state == SLEEPING){
                p->state = RUNNABLE;
            }
        }
        release(&p->lock);
    }
    return 0;
}

```

kexec()

kexec() 은 thread group의 leader만 호출할 수 있고, leader가 호출할 경우 page table을 swap하기 전 살아있는 sibling을 모두 drain해야 한다는 점에서 kexit() 함수의 group leader part와 동일하게 구현하였다.

```

int
kexec(char *path, char **argv)
{
    char *s, *last;
    int i, off;
    uint64 argc, sz = 0, sp, ustack[MAXARG], stackbase;
    struct elfhdr elf;
    struct inode *ip;
    struct proghdr ph;
    pagetable_t pagetable = 0, oldpagetable;
    struct proc *p = myproc();

    // Is this group leader?
    // if not, return -1. (non-group leader must not call exec() function)
    if(p->tgid != p->pid){
        return -1;
    }

    acquire(&wait_lock);

    // before switching page table,
    // group leader should drain all sibling threads.
    // 1st. mark kill for all siblings.
    struct proc* pp;
    for(pp = proc; pp < &proc[NPROC]; pp++){
        acquire(&pp->lock);
        if(pp->state != UNUSED && pp->tgid == p->pid && pp != p){
            pp->killed = 1;
            if(pp->state == SLEEPING){
                pp->state = RUNNABLE;
            }
        }
        release(&pp->lock);
    }

    // 2nd. infinitely loop until all siblings terminate.
    for(;;){
        int hasesiblings = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++){
            acquire(&pp->lock);
            if(pp->tgid == p->pid && pp != p && pp->state != UNUSED){
                if(pp->state == ZOMBIE){
                    freeproc(pp);
                }
                else hasesiblings = 1;
            }
            release(&pp->lock);
        }

        if(hasesiblings == 0) break;

        sleep(p->group_leader, &wait_lock);
    }

    release(&wait_lock);

    // below is legacy xv6 code ... -----

```

Results



Results

Document the compilation and execution process. Include screenshots verifying that all requirements are met, alongside explanations of the execution flow.

[proj2] 간단한 test용 코드입니다. #47

JngVedere started this conversation in test-cases



JngVedere 2 days ago

1. 테스트 목적 (Objective)

간단한 스레드 기능들을 검증합니다.
panic, 데드락 혹은 usertrap에서의 unexpected scause 에러 등이 발생하면 안 됩니다.

test1. 스레드 생성 및 소멸을 확인합니다.
test2. 최대 생성 한계치를 받아하는지 검사합니다.
test3. 스레드간 메모리 공유가 잘 되는지 확인합니다. (page table 공유 확인)
test4. 스트레스 및 메모리 누수 테스트 목적입니다.
test5. 그룹 리더 종료 시 제대로 thread들이 종료되는지 확인합니다.
test6. 그룹 리더와 파생 스레드가 exec 실행 시 어떻게 동작하는지 확인합니다.
test7. kill이 스레드 전원을 죽이는지 확인합니다.
test8. wait가 그룹 리더만을 받는지 확인합니다.
test9. 멀티코어 환경에서의 스트레스 테스트입니다. (test4와 비슷함)

naive하게 만든 거라 개선 사항 있으면 말씀해주세요.

Github Discussion에 올라온 테스트를 실행하여 actual output이 expected output과 동일함을 확인하였다.

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ threadtest

=== TEST 1. THREAD CREATION/JOIN TEST ===
THREAD CREATED PID : 4
THREAD JOINED PID : 4
TEST 1 PASS

=== TEST 2. MAXIMUM THREAD TEST ===
TEST 2 PASS

=== TEST 3. SHARED MEMORY TEST ===
TEST 3 PASS. SHARED VALUE : 50

=== TEST 4. THREAD CREATION STRESS TEST ===
CREATING THREADS...
100 THREADS CREATED AND JOINED
200 THREADS CREATED AND JOINED
300 THREADS CREATED AND JOINED
400 THREADS CREATED AND JOINED
500 THREADS CREATED AND JOINED
600 THREADS CREATED AND JOINED
700 THREADS CREATED AND JOINED
800 THREADS CREATED AND JOINED
900 THREADS CREATED AND JOINED
1000 THREADS CREATED AND JOINED
TEST 4 PASS.

=== TEST 5. GROUP LEADER EXIT TEST ===
TEST 5 PASS. create_count : 61

=== TEST 6. THREAD GROUP EXEC TEST ===
[PASS] THREAD EXEC FAILED AS EXPECTED
[PASS] LEADER EXEC PASSED.
[PASS] EXEC RESOURCE MANAGEMENT

=== TEST 7. KILL TEST ===
[PASS] KILL TEST PASSED

=== TEST 8. WAIT TEST ===
[PASS] SUCCESSFULLY WAITED FOR GROUP LEADER PROCESS

=== TEST 9. CONCURRENCY STRESS TEST ===
[PASS] CONCURRENCY SBRK TEST PASSED. NO KERNEL PANIC.
$
```

Troubleshooting



Troubleshooting

If you encountered any issues during the assignment, describe the issues and your resolution process. For unresolved issues, explain what they were and what approaches you attempted.


```

=== TEST 5. GROUP LEADER EXIT TEST ===
1 (tgid=1 tf_va=3ffffffe000) sleep init
2 (tgid=2 tf_va=3ffffffe000) sleep sh
2269 (tgid=2269 tf_va=3ffffffe000) run threadtest
3292 (tgid=3292 tf_va=3ffffffe000) zombie threadtest
3293 (tgid=3293 tf_va=3ffffffe000) zombie threadtest
3294 (tgid=3294 tf_va=3ffffffe000) zombie threadtest
3295 (tgid=3295 tf_va=3ffffffe000) zombie threadtest
3296 (tgid=3296 tf_va=3ffffffe000) zombie threadtest
3297 (tgid=3297 tf_va=3ffffffe000) zombie threadtest
3298 (tgid=3298 tf_va=3ffffffe000) zombie threadtest
3299 (tgid=3299 tf_va=3ffffffe000) zombie threadtest
3300 (tgid=3300 tf_va=3ffffffe000) zombie threadtest
3301 (tgid=3301 tf_va=3ffffffe000) zombie threadtest
3302 (tgid=3302 tf_va=3ffffffe000) zombie threadtest
3303 (tgid=3303 tf_va=3ffffffe000) zombie threadtest
3304 (tgid=3304 tf_va=3ffffffe000) zombie threadtest
3305 (tgid=3305 tf_va=3ffffffe000) zombie threadtest
3306 (tgid=3306 tf_va=3ffffffe000) zombie threadtest
3307 (tgid=3307 tf_va=3ffffffe000) zombie threadtest
3308 (tgid=3308 tf_va=3ffffffe000) zombie threadtest
3309 (tgid=3309 tf_va=3ffffffe000) zombie threadtest
3310 (tgid=3310 tf_va=3ffffffe000) zombie threadtest
3311 (tgid=3311 tf_va=3ffffffe000) zombie threadtest
3312 (tgid=3312 tf_va=3ffffffe000) zombie threadtest
3313 (tgid=3313 tf_va=3ffffffe000) zombie threadtest
3314 (tgid=3314 tf_va=3ffffffe000) zombie threadtest
3315 (tgid=3315 tf_va=3ffffffe000) zombie threadtest
3316 (tgid=3316 tf_va=3ffffffe000) zombie threadtest
3317 (tgid=3317 tf_va=3ffffffe000) runble threadtest
3318 (tgid=3318 tf_va=3ffffffe000) runble threadtest
3319 (tgid=3319 tf_va=3ffffffe000) runble threadtest
3320 (tgid=3320 tf_va=3ffffffe000) used
TEST 5 FAILED. create_count : 28

```

앞선 usertest의 TEST 5에서, thread_join()에 의한 sub-thread 회수는 잘 되었지만 fork()를 통해 얻어지는 single-thread process가 proc table에 들어가지 못함을 procdump() 함수를 통해 확인하였다. create_count가 28에서 멈추었다는 것은 29번째 fork에서 fork fail이 되어 usertest의 TEST 5가 멈추었다는 뜻이므로, kclone()의 fork fail에 대한 디버깅 프린트를 각 경우에 대하여 출력하도록 했다.

```

// === fork path ===
if ((np->mm = mm_alloc()) == 0)
    goto fail;
if ((np->mm->pagetable = proc_pagetable(p: np)) == 0)
    goto fail;
if (uvmcopy(p->mm->pagetable, np->mm->pagetable, p->mm->sz) < 0)
    goto fail;

```

여기서 uvmcopy가 실패해서 fork fail이 일어났음을 알게 되었고, 더 나아가 uvmcopy 안의 kalloc() 함수가 실패했음을 확인했다.

```

int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i;
    uint flags;
    char *mem;

    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walk(pagetable: old, va: i, alloc: 0)) == 0)
            continue; // page table entry hasn't been allocated
        if((*pte & PTE_V) == 0)
            continue; // physical page hasn't been allocated
        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);
        if((mem = kalloc()) == 0){
            printf("fail.....\n");
            goto err;
        }
        memmove(mem, (char*)pa, PGSIZE);
        if(mappages(pagetable: new, va: i, size: PGSIZE, pa: (uint64)mem, perm: flags) != 0){
            kfree(mem);
            goto err;
        }
    }
    return 0;

err:
    uvmunmap(pagetable: new, va: 0, npages: i / PGSIZE, do_free: 1);
    return -1;
}

```

uvmcopy() 내부 kalloc이 실패했을 때 fail을 출력하도록 했다.

이는 zombie 상태에 들어간 자식 프로세스들이 부모에 의해 free되기 전 fork()가 계속해서 호출되어 zombie가 된 스레드들이 proc table에서 계속해서 유지되고 있다고 판단하여, “어차피 free가 될 프로세스들이라면 미리 공간을 확보할 수 있게끔 trapframe mapping을 선제적으로 끊자”는 생각 하에 free 이전 uvmunmap을 호출하여 proc table의 공간을 만들어주었고, 이 방법으로 TEST 5를 통과할 수 있었다.

```

// preemptive unmapping.
if(p->mm && p->group_leader == p){
    acquire(&p->mm->lock);
    if(p->mm->refcount == 1){
        uvmunmap(p->mm->pagetable, 0, p->mm->sz / PGSIZE, 1);
        p->mm->sz = 0;
    }
    release(&p->mm->lock);
}
}

```